



TRAKYA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BLM421 PROJE II
PROJE RAPORU

PROJE ADI:

Lojistik Veri Temelli Rota ve Depo Optimizasyonu

PROJE EKİBİ:

1221602083 - Ferhat ÖZGEN

1221602008 - Furkan EROĞLU

1221602088 - Pınar KARABULUT

1231602901 - Kerem Salih TURGAY

1221602029 - Umut HATA

PROJE DANIŞMANI:

Dr. Öğr. Üyesi Turgut DOĞAN

Edirne - 2026

İÇİNDEKİLER

1. GİRİŞ & PROJENİN TANITIMI	1
2. GEREKSİNİM ANALİZİ	4
3. TASARIM	6
4. GELİŞTİRME	12
5. TEST ve DOĞRULAMA	16
6. SONUÇ	20
7. GitHub BAĞLANTISI	21
KAYNAKLAR	22

1. GİRİŞ & PROJENİN TANITIMI

Küresel e-ticaret hacminin hızla büyümesi ve tüketici beklentilerinin giderek yükselmesiyle birlikte lojistik sektörü, son yıllarda köklü bir dönüşüm sürecine girmiştir. Önümüzdeki yıllarda küresel lojistik pazarının hızlı büyümesini sürdüreceği öngörülmekte; bu büyümenin temel itici gücü olarak teslimat verimliliği ve dinamik depo yönetimi gösterilmektedir. Bu bağlamda, geleneksel lojistik karar destek sistemlerinin yetersizlikleri giderek daha belirgin bir biçimde ortaya çıkmaktadır.

Geleneksel depo ve dağıtım merkezi konumlandırma yaklaşımları, ağırlıklı olarak statik coğrafi analizlere, uzman deneyimine ve dönemsel planlama döngülerine dayanmaktadır. Bu yaklaşımlar; meteorolojik değişkenler, saatlik talep dalgalanmaları ve özel etkinlik takvimi gibi dinamik faktörleri gerçek zamanlı olarak işleme kapasitesinden yoksundur. Söz konusu yetersizlik, operasyonel maliyetlerin artmasına, müşteri memnuniyetinin düşmesine ve rekabet avantajının kaybedilmesine doğrudan zemin hazırlamaktadır.

Bu çalışmada geliştirilen "Lojistik Veri Temelli Rota ve Depo Optimizasyonu Modeli", söz konusu açığı kapatmak amacıyla bütünleşik bir veri bilimi mimarisi üzerine inşa edilmiştir. Sistem; coğrafi koordinatları, hava durumunu, teslimat zaman damgalarını ve sipariş hacmi bilgilerini tek bir analitik çatı altında bir araya getirmektedir. Makine öğrenmesi tabanlı kümeleme algoritmaları ve derin öğrenme destekli zaman serisi tahmin modelleri aracılığıyla sistem, statik kural tabanlı yaklaşımların ötesine geçerek veriyi öğrenen, örüntüleri keşfeden ve dinamik öneriler üreten bir karar destek altyapısı sunmaktadır.

Projenin özgün katkısı üç temel boyutta değerlendirilebilir: Birincisi, lojistik verilerinin çok değişkenli ve gerçek zamanlı işlenmesini mümkün kılan bir veri akış mimarisinin tasarlanması; ikincisi, KMeans kümeleme algoritması ile LSTM derin öğrenme modelinin tek bir pipeline altında bütünleştirilmesi; üçüncüsü ise gerçek dünya lojistik davranışlarını simüle eden ve sistemin farklı coğrafi senaryolarda test edilmesine olanak tanıyan dijital ikiz tabanlı sentetik veri üretim modülünün geliştirilmesidir.

1.1 Problem Tanımı ve Motivasyon

Lojistik operasyonlarında depo konumunun belirlenmesi, operasyonel maliyeti ve teslimat sürelerini doğrudan etkileyen en kritik stratejik kararlar arasında yer almaktadır. Lojistik maliyetlerinin önemli bir bölümünün depolama ve dağıtım ağı verimsizliklerinden kaynaklandığı bilinmektedir. Mevcut sistemlerin büyük bölümü, depo ağlarını yıllık veya dönemsel planlamalar çerçevesinde, sabit lokasyonlara göre yönetmektedir. Bu statik yaklaşım; nüfus hareketliliği, kentsel dönüşüm projeleri ve tüketim alışkanlıklarındaki hızlı değişimler karşısında ciddi bir uyum sorunu yaratmaktadır.

Problem daha spesifik bir biçimde ele alındığında, mevcut sistemlerin dört temel eksikliği öne çıkmaktadır. İlk olarak, statik lokasyon yapısı; depo ağlarının nüfus hareketliliği ve yeni yerleşim bölgelerine göre güncellenmesini güçleştirmektedir. İkinci olarak, zaman boyutunun göz ardı edilmesi; talebin gün içindeki iş merkezleri ile yerleşim bölgeleri arasında yer değiştirmesine rağmen, sistemin bu dinamik farkı tanıyamamasına neden olmaktadır. Üçüncü olarak, yatırım kararlarının anlık müşteri davranışı yerine uzun vadeli ve esneklikten yoksun stratejik varsayımlara dayandırılması, kaynakların verimsiz kullanılmasına zemin hazırlamaktadır. Son olarak, rota optimizasyonunda hava durumu ve genel müşteri davranış modellemesinin yetersiz entegrasyonu, gereksiz karbon emisyonlarına ve artan operasyonel maliyetlere yol açmaktadır.

Yapılan literatür incelemesinde, lojistik optimizasyonuna yönelik çeşitli çalışmaların mevcut olduğu görülmektedir. Farahani ve Hekmatfar [1] tarafından derlenen kapsamlı çalışmada, tespit-tahsis (facility location) problemleri üzerine geliştirilen modeller sistematik biçimde ele alınmakta; ancak bu modellerin büyük çoğunluğunun statik veri yapısına dayandığı ve dinamik talep değişkenliğini modelleyemediği vurgulanmaktadır. Benzer biçimde, Chopra ve Meindl [2], tedarik zinciri yönetiminde dinamik kapasite planlamasının kritik önemine dikkat çekmekle birlikte, gerçek zamanlı çok değişkenli veri entegrasyonuna ilişkin uygulamalı ve ölçeklenebilir bir çözüm önerememektedir. Öte yandan, derin öğrenme tabanlı talep tahmini literatürü incelendiğinde, Hochreiter ve Schmidhuber [3] tarafından önerilen LSTM mimarisinin zaman serisi tahmininde geleneksel ARIMA ve üstel düzleştirme yöntemlerine kıyasla belirgin biçimde üstün performans sergilediği görülmektedir. Bu proje; söz konusu literatür boşluğunu kapatmak, mevcut yaklaşımların sınırlılıklarını aşmak ve lojistik karar destek sistemlerine bütünlük, veri odaklı ve dinamik bir çözüm kazandırmak amacıyla tasarlanmıştır.

1.2 Projenin Amacı ve Kapsamı

Projenin temel amacı; tarihsel ve simüle edilmiş teslimat verileri, meteorolojik koşullar, trafik yoğunluğu endeksi ve zaman serisi bilgisi kullanılarak lojistik operasyonlarında dinamik depo konumu önerisi ve rota optimizasyonu gerçekleştirebilen, uçtan uca çalışan bir veri odaklı karar destek sistemi geliştirmektir. Sistem; veri üretimi, depolama, ön işleme, modelleme ve görselleştirme aşamalarını bütünleşik bir mimari içinde ele almakta ve her bir aşamanın bağımsız olarak geliştirilebileceği modüler bir yapı sunmaktadır. Bu amaç doğrultusunda proje kapsamında aşağıdaki alt hedefler gerçekleştirilmiştir.

- 25.000 satırlık içerisinde hem gerçek hem de sentetik ama gerçekçi lojistik verisinin üretilmesi sağlanmış olup Edirne merkez ili üzerinden veri seti oluşturulması
- K-Means kümeleme algoritması aracılığıyla teslimat yoğunluk bölgelerinin tespit edilmesi ve optimal depo lokasyonlarının önerilmesi,
- LSTM tabanlı derin öğrenme modeliyle saatlik ve günlük talep tahmininin gerçekleştirilmesi,
- Keşifsel veri analizi yoluyla teslimat örüntülerinin, yoğunluk haritalarının ve istatistiksel dağılımların görselleştirilmesi,
- Tüm modüllerin beraber çalışabildiği bir ortam altında birleştirilerek uçtan uca çalışan bir sistem oluşturulması.

1.3 Projenin yenilikçi Yönü

Mevcut lojistik optimizasyon sistemleriyle karşılaştırıldığında, bu projenin öne çıkan yenilikçi yönleri şu şekilde özetlenebilir:

- **Çok değişkenli veri entegrasyonu:** Yalnızca coğrafi koordinatlar değil; hava durumu, sipariş hacmi ve özel gün bilgisi gibi çok boyutlu değişkenler eş zamanlı olarak modellenmektedir.
- **Modüler mimari:** Her bir bileşen (veri üretimi, ön işleme, modelleme, analiz) bağımsız olarak geliştirilebilir ve güncellenebilir yapıda tasarlanmıştır.

2. GEREKSİNİM ANALİZİ

2.1 Platform ve İşletim Sistemleri Gereksinimleri

Sistem, masaüstü ortamında çalışacak biçimde tasarlanmıştır. Python 3.10 ve üzeri bir yorumlayıcı ve gerekli Python kütüphanelerinin kurulu olduğu herhangi bir işletim sistemi (Windows 10/11, macOS 12+, Ubuntu 20.04+) üzerinde çalışabilmektedir.

2.2. Çözülen Problemler ve Hedef Kitle

Proje, lojistik sektöründe karşılaşılan üç temel probleme çözüm üretmektedir. Birincisi, depo konumlandırmasında statik ve sezgisel karar verme sürecidir; geliştirilen sistem, veri odaklı kümeleme analizi aracılığıyla bu süreci nesnel ve tekrarlanabilir bir yapıya kavuşturmaktadır. İkincisi, talep tahminindeki yetersizliktir; LSTM modeli sayesinde saatlik ve günlük sipariş hacmi tahminleri yapılabilen, bu sayede proaktif kapasite planlaması mümkün hale gelmektedir. Üçüncüsü ise operasyonel verimsizliktir; sistemimiz tarafından tasarlanan rota optimizasyonu, teslimat maliyetlerinin düşürülmesine katkı sağlamaktadır.

Sistemin birincil hedef kitlesi lojistik ve kargo şirketleridir. Bununla birlikte, sipariş yoğunluğuna göre dinamik depo konumlandırması ihtiyacı duyan e-ticaret şirketleri de önemli bir kullanıcı grubu olarak değerlendirilmektedir. Yüksek sipariş hacmine sahip, çok noktalı dağıtım ağı işleten her ölçekten kuruluşun sistemden faydalanabileceği öngörülmektedir.

2.3. Mevcut Çözümlerden Farkı

Piyasada mevcut olan lojistik optimizasyon yazılımları (örn. Oracle Transportation Management, SAP TM) büyük ölçüde kurumsal düzeyde lisanslı ve yüksek maliyetli çözümler sunmaktadır. Bu sistemlerin küçük ve orta ölçekli işletmelere erişimi oldukça sınırlı kalmaktadır. Öte yandan açık kaynaklı çözümler incelendiğinde, çok değişkenli dinamik analiz ile derin öğrenme entegrasyonunu bir arada sunan bütünsel bir sistemin literatürde yeterince yer bulmadığı görülmektedir. Bu proje, söz konusu boşluğu açık kaynak araçlarla ve akademik bir yaklaşımla doldurmayı hedeflemektedir.

2.4. Kullanılan Teknolojiler ve Gerekçeleri

Projede kullanılan teknolojiler ve bu teknolojilerin tercih edilme gerekçeleri Çizelge 2.1'de özetlenmektedir.

Çizelge 2.1. Kullanılan teknolojiler, avantaj ve dezavantajları

Teknoloji	Kullanım Amacı
Python 3.12	Ana programlama dili
Pandas / NumPy	Veri işleme ve sayısal hesaplama
Scikit-learn	K-Means kümeleme ve ön işleme
Keras / TensorFlow	LSTM derin öğrenme modeli
Matplotlib / Seaborn	Veri görselleştirme
Folium	Coğrafi harita görselleştirme
python-dotenv	Ortam değişkeni yönetimi
meteostat	Edirne hava durumu verilerini tarihe göre çekmek
osmnx	Edirne merkezin coğrafi konumlarını çekmek
random	Sentetik veri oluşturma
Git / GitHub	Versiyon kontrolü ve iş birliği

2.5. Kaynak Kısıtları ve Çözüm Önerileri

Projenin geliştirilmesi sürecinde çeşitli kaynak kısıtlarıyla karşılaşmıştır. Gerçek dünya lojistik verilerine erişimin kısıtlı olması nedeniyle birtakım verilerin eldesi kısmında gerçekçi dağılım parametreleri baz alınarak sentetik veri üretimi yapılmıştır. Bu yaklaşım, sistemin farklı senaryolarda test edilmesine de olanak tanımıştır. Veri tabanı bağlantısının ekip üyeleri

arasında standart hale getirilmesi amacıyla bağlantı bilgileri .env dosyasında yönetilmekte ve bu dosyanın versiyon kontrol sistemine dahil edilmesi .gitignore mekanizmasıyla engellenmektedir.

2.6. Geliştirme Sürecinde Kullanılan Destek Araçları

Projenin geliştirilmesi sürecinde, doğrudan proje çıktısı olmamakla birlikte geliştirme verimliliğini artıran çeşitli araçlardan yararlanılmıştır. Versiyon kontrolü ve ekip içi iş birliği için Git ve GitHub platformu kullanılmıştır. Geliştirme ortamı olarak PyCharm tercih edilmiş; Python bağımlılıklarının yönetimi için pip ve sanal ortam (venv) mekanizmaları kullanılmıştır. Ortam değişkenlerinin güvenli biçimde yönetimi için python-dotenv kütüphanesinden yararlanılmıştır.

3. TASARIM

3.1 Genel Sistem Mimarisi

Sistem, birbirinden bağımsız ancak birbirine entegre beş ana katmandan oluşmaktadır. Bu katmanlar sırasıyla veri eldesi, veri oluşturma, veri birleştirme, modelleme, analiz ve görselleştirme olarak tanımlanmıştır.

En alt katmanda veri kaynakları yer almaktadır. Bu katmanda Meteostat API'sinden alınan gerçek saatlik Edirne hava verisi ile OSMnx kütüphanesi aracılığıyla OpenStreetMap'ten elde edilen Edirne yol ağı düğümleri bir araya getirilmektedir. Veri işleme katmanında, ham veriler feature_engine modülü aracılığıyla işlenmekte; trafik indeksi, sipariş hacmi ve özel gün bilgisi hesaplanarak orders_history.csv dosyası üretilmektedir. Modelleme katmanında iki paralel süreç yürütülmektedir: LSTM modeli günlük bölgesel sipariş sayısı tahmini yaparken Kmeans algoritması optimal depo konumlarını belirlemektedir. Analiz katmanında keşifsel veri analizi gerçekleştirilmekte, sonuçlar görselleştirme katmanında statik grafiklerde gösterilmiştir.

3.2 Veri Üretim Modülü

Projenin en özgün bileşenlerinden biri, gerçek coğrafi ve meteorolojik veriye dayanan sentetik veri üretim modülüdür. Bu modül üç aşamalı bir süreç izlemektedir.

Birinci aşamada `edirne_weather.py` betiği çalıştırılarak Meteostat API'si aracılığıyla Ocak 2025 - Mayıs 2026 dönemine ait saatlik hava verileri çekilmektedir. Sıcaklık, yağış, rüzgar hızı ve hava kodu (coco) bilgileri içeren bu veri seti, üç kategorili bir hava etiketi sistemine dönüştürülmektedir: 0 (açık), 1 (yağmur/sis) ve 2 (kar/fırtına).

İkinci aşamada `extract_edirne_nodes.py` betiği çalıştırılarak OSMnx kütüphanesi ile Edirne'nin sürüş ağındaki tüm sokak düğümleri indirilmektedir. Şehrin tamamındaki düğümlerden, Çizelge 3.1'de listelenen on stratejik noktaya 1.2-1.5 km mesafe içinde kalanlar filtrelenerek `edirne_nodes.csv` dosyasına kaydedilmektedir. Bu yaklaşım, teslimat noktalarının gerçek dışı veya ıssız bölgelere düşmesini engellemektedir.

İkinci aşamada belirlenen stratejik bölgeler, sistemin veri üretim mantığının temelini oluşturmaktadır. Saraçlar ve Ayşekadın gibi ticari merkezler hem trafik yoğunluğunu hem de sipariş hacmini artıran bölgeler olarak tanımlanmıştır. Delta bölgesi ise öğrenci yurdu sayısı bakımından şehirdeki en yüksek sipariş yoğunluğuna sahip bölgelerden biri olarak sisteme dahil edilmiştir. Kırkpınar Haftası, dini bayramlar, sınav haftası ve resmi tatil günleri, sipariş hacmi arttıracak biçimde modellenmiştir.

Üçüncü aşamada `generate_history.py` betiği hava ve sokak ağı verilerini birleştirmektedir. Her saatlik zaman dilimi için Poisson dağılımıyla sipariş sayısı belirlenmekte, bu siparişler için stratejik bölgelere yakınlık ağırlıkları hesaplanarak gerçek sokak düğümleri arasından teslimat lokasyonları seçilmektedir. Ardından `feature_engine` modülü devreye girerek her teslimat noktası için trafik indeksi, sipariş hacmi ve özel gün bilgisi hesaplanmaktadır.

3.3 Demand Engine Modülü

Projenin proaktif karar mekanizmasını besleyen ve derin öğrenme modelinin eğitim altyapısını oluşturan `demand_engine.py` modülü, Edirne merkezindeki kargo ve lojistik hareketliliğini saatlik çözünürlükte simüle eden gelişmiş bir dinamik veri üretim motorudur. Geçmiş hava durumu verilerini (`edirne_weather_2025_2026.csv`) temel alan bu motor; zaman, takvim anomalileri ve mahalle bazlı sosyo-ekonomik profilleri harmanlayarak makine öğrenmesi modellerinin can damarı olan zamansal örüntüleri içeren yüksek kaliteli bir zaman

serisi veri seti (hourly_demand.csv) inşa eder. Öznitelik mühendisliği kapsamında sistem; resmî tatillerin yanı sıra Edirne'ye özgü Tarihi Kırkpınar Yağlı Güreşleri Haftası'nı (23-30 Haziran) ve Trakya Üniversitesi'nin sömestr ile yaz tatili dönemlerini tam kronolojik tarihleriyle takip eder. Geleneksel simülasyonlardan farklı olarak bu modül, üniversitenin Mühendislik, Tıp ve Diş Hekimliği fakültelerinin vize ve final sınav takvimlerini ayrı ayrı işlerken, sınavların hemen öncesindeki 1 haftalık süreçleri kütüphaneye kapanma ve lojistik davranış değişimlerini temsil eden "Hazırlık Haftası" (is_prep_week) özniteliğiyle dinamik olarak sisteme gömer.

3.4 LSTM Talep Tahmin Modeli

Saatlik sipariş hacminin tahmin edilmesi amacıyla LSTM (Long Short-Term Memory) mimarisi tercih edilmiştir. LSTM, standart tekrarlayan sinir ağlarının uzun vadeli bağımlılıkları öğrenememesi sorununu gidermek üzere Hochreiter ve Schmidhuber [3] tarafından geliştirilmiş olup zaman serisi tahmininde yaygın biçimde kullanılmaktadır.

Model mimarisi iki katmanlı LSTM yapısından oluşmaktadır. İlk LSTM katmanı 50 birimden oluşmakta ve sonraki katmana dizi çıktısı vermektedir. Her iki LSTM katmanından sonra %20 oranında Dropout uygulanarak aşırı öğrenme riski azaltılmaktadır. Çıkış katmanına doğru iki tam bağlı (Dense) katman yer almakta; son Dense katmanı tek bir değer (bir sonraki gün için oluşturulacak toplam sipariş hacmi) üretmektedir.

Veri ön işleme aşamasında ham teslimat verisi saatlik gruplara yeniden örneklenmekte; sipariş hacmi, trafik indeksi, hava durumu etiketi ve özel gün bilgisi MinMaxScaler ile [0,1] aralığına normalize edilmektedir. Kayan pencere (sliding window) yöntemiyle son 24 saatin verisi girdi olarak kullanılmakta ve bir sonraki saatin sipariş hacmi tahmin edilmektedir. Veri setinin %80'i eğitim, %20'si test amacıyla ayrılmıştır. Model Adam optimizier ve ortalama karesel hata (MSE) kayıp fonksiyonu ile 20 epoch boyunca eğitilmektedir.

3.5 Dinamik Hub Konumu Optimizasyon Modeli (K-means)

Geleneksel sabit depolama maliyetlerini ortadan kaldırmak ve değişken kargo taleplerine esnek çözümler sunmak amacıyla, optimal geçici dağıtım merkezi konumlarının belirlenmesinde K-Means Kümeleme algoritması temel alan proaktif-reaktif hibrit bir optimizasyon modeli kullanılmıştır. Bu model, rijit ve durağan kümeleme mimarileri yerine,

yönetim paneli üzerinden dinamik olarak değiştirilebilen araç/konteyner kapasite kısıtlarına tam uyum sağlayacak esnek bir yapıda tasarlanmıştır.

Optimizasyon süreci iki temel aşamadan oluşmaktadır:

Proaktif Aşama (Dinamik K Seçimi): Sistem öncelikle LSTM modelinden gelen ertesi güne ait mahalle bazlı kargo tahminlerini toplar. Elde edilen toplam tahmini paket hacmi, yönetici panelinden girilen dinamik hub kapasite kısıtına bölünerek açılması gereken en uygun geçici hub sayısı bir üst tam sayıya yuvarlanacak şekilde sezgisel olarak hesaplanır:

Reaktif Aşama (Coğrafi Kümeleme): O sabah Edirne sokaklarında kesinleşen gerçek siparişlerin enlem ve boylam koordinatları (lat, lon), hesaplanan dinamik değeriyle K-Means algoritmasına girdi olarak verilir. Algoritmanın başlangıç merkezlerini rassal seçerek yerel minimuma takılmasını önlemek amacıyla K-Means++ başlatma mimarisi tercih edilmiş; model $n_init=10$ parametresiyle en kararlı ağırlık merkezlerini bulana kadar eğitilmiştir. Kurulan küme yapısının sonucunda elde edilen matematiksel centroid değerleri, o gün Edirne genelinde kurulacak geçici mobil hub istasyonlarının tam coğrafi koordinatlarını belirler

3.6 Görselleştirme Modülü

Sistem, analiz sonuçlarını iki farklı formatta sunmaktadır. EDA görselleri Matplotlib ve Seaborn kütüphaneleri kullanılarak PNG formatında üretilmekte ve results/ klasörüne kaydedilmektedir.

3.7 Proje Dizin Yapısı

Projenin modüler dizin yapısı Çizelge 3.1'de özetlenmektedir. Her klasör belirli bir sorumluluk alanına karşılık gelecek biçimde tasarlanmıştır.

Çizelge 3.1. Proje dizin yapısı ve açıklamaları

Dizin / Dosya	İçerik ve Sorumluluk
data/	Ham ve işlenmiş veri dosyaları (CSV formatında)
models/saved/	Eğitilmiş LSTM modeli (.h5) ve scaler (.pkl) dosyaları
results/	Modelin eğitim sürecine ait kayıp grafikleri ve gerçek-tahmin karşılaştırma analiz çıktıları
simulation/	Sentetik veri üretim hattı, coğrafi talep üretim motoru ve mahalle bazlı sosyo-ekonomik profil tanımları.
src/forecasting/	LSTM veri ön işleme modülü
src/warehouse/	dinamik K-Means hub optimizasyon motoru ve kurye rota planlama bileşenleri
results/	Üretilen grafik ve harita çıktıları
.env	Veritabanı bağlantı bilgileri (Git'e dahil edilmez)

4. GELİŞTİRME

4.1. Geliştirme Ortamı ve Süreç Yönetimi

Projenin geliştirme süreci, modüler ve paralel çalışmaya olanak tanıyan bir iş bölümü çerçevesinde yürütülmüştür. Tüm kod tabanı Git versiyon kontrol sistemi ile yönetilmiş; ekip üyeleri GitHub üzerinden ortak bir repoya katkıda bulunmuştur. Geliştirme ortamı olarak Pycharm tercih edilmiş, Python bağımlılıkları sanal ortam (venv) ve pip aracılığıyla yönetilmiştir. Ortam değişkenleri python-dotenv kütüphanesi ile .env dosyasında saklanmış; bu dosyanın versiyon kontrol sistemine dahil edilmesi .gitignore mekanizmasıyla engellenmiştir.

Geliştirme süreci beş ana aşamada ilerlemiştir: veri altyapısının kurulması, talep simülasyon motorunun geliştirilmesi, LSTM modelinin eğitilmesi, optimizasyon motorlarının yazılması ve tüm modüllerin pipeline altında birleştirilmesi. Her aşama bağımsız olarak test edilmiş ve bir sonraki aşamaya geçmeden önce doğrulanmıştır.

4.2. Veri Altyapısının Geliştirilmesi

Veri altyapısı iki paralel süreçle oluşturulmuştur. Birinci süreçte `extract_edirne_nodes.py` betiği OSMnx kütüphanesi aracılığıyla OpenStreetMap'ten Edirne'nin tüm sürüş ağı düğümleri indirilmiştir. Ardından `order_sampler.py` içindeki `cluster_nodes_by_district` fonksiyonu, her sokak düğümünü beş mahalle merkezine olan Haversine mesafesine göre sınıflandırmış; merkeze 4 km'den uzak düğümler lojistik alan dışı kabul edilerek elenmiştir.

İkinci süreçte `edirne_weather.py` betiği Meteostat API'si üzerinden Ocak 2025 - Mayıs 2026 dönemine ait saatlik hava verileri çekmiştir. Sıcaklık, yağış ve rüzgar bilgileri üç kategorili bir etikete dönüştürülmüştür: 0 (açık hava), 1 (yağmur/sis) ve 2 (kar/fırtına). Bu iki veri kaynağı `demand_engine.py` modülünde birleştirilerek `hourly_demand.csv` dosyası üretilmiştir.

4.3. Talep Simülasyon Motorunun Geliştirilmesi

Projenin en özgün geliştirme bileşeni olan talep simülasyon motoru, `district_profiles.py` ve `demand_engine.py` modüllerinden oluşmaktadır. `district_profiles.py` dosyasında Edirne'nin beş mahallesi (Saraçlar, Balkan, Karaağaç, Şükrüpaşa, Ayşekadın) için 24 saatlik aktivite eğrileri, hafta sonu çarpanları ve trafik faktörleri tanımlanmıştır. Balkan ve Ayşekadın bölgeleri öğrenci yurdu yoğunluğu nedeniyle gece saatlerinde yüksek talep gösteren profillere sahipken, Saraçlar ticari merkez karakterine uygun gündüz ağırlıklı bir aktivite eğrisi sergilemektedir.

`demand_engine.py` içindeki `get_calendar_features` fonksiyonu, Trakya Üniversitesi'nin üç fakültesine (Mühendislik, Tıp Fakültesi, Diş Hekimliği) ait sınav takvimlerini, sömestr ve yaz tatillerini ve sınav öncesi hazırlık haftalarını ayrı ayrı modellemektedir. Bu yapı sayesinde sistem; sınav dönemlerinde Balkan ve Ayşekadın'da sipariş artışını, tatil dönemlerinde ise aynı bölgelerde ciddi oranında düşüşü gerçekçi biçimde simüle edebilmektedir. Kırkpınar Yağlı Güreş Festivali (Haziran sonu), Cumhuriyet Bayramı ve dini bayramlar gibi özel günler de takvime dahil edilmiştir.

Saatlik talep hesaplaması çok bileşenli bir çarpım formülüne dayanmaktadır. Baz talep değeri; saatlik aktivite çarpanı, hafta sonu çarpanı, hava durumu çarpanı, özel gün çarpanı ve akademik takvim çarpanının çarpımı ile belirlenmektedir. Buna ek olarak %3 olasılıkla 1.5-2.2 arasında rastgele bir ani sipariş patlaması ve Gauss gürültüsü eklenerek gerçek dünya lojistik verisine özgü stokastik yapı korunmaktadır.

`data_pipeline.py` modülü ise sistemin zaman içinde ileri taşınabilmesini sağlayan akıllı bir veri köprüsü işlevi görmektedir. Veri setinde boşluk tespit edildiğinde (örneğin gelecek günlere ait tahmin talebi geldiğinde) `demand_engine` kuralları çalıştırılarak eksik günler otomatik olarak doldurulmaktadır.

4.4. LSTM Modelinin Geliştirilmesi ve Eğitimi

LSTM modelinin geliştirilmesi `preprocess.py`, `train_lstm.py` ve `evaluate_lstm.py` modülleri aracılığıyla gerçekleştirilmiştir. `preprocess.py` içindeki `prepare_multi_output_lstm_data` fonksiyonu, saatlik talep verisini 24 adımlık kayan pencere yöntemi ile LSTM'e uygun üç boyutlu tensör formatına ([örnekler, zaman adımı, özellik sayısı]) dönüştürmektedir. Girdi

özellikleri beş mahalle talebi ile dokuz bağlamsal değişkenden (hava durumu, hafta sonu, özel gün, sömestr tatili, yaz tatili, hazırlık haftası, üç fakülte sınav bayrağı) oluşmakta; toplam 14 özellik MinMaxScaler ile [0,1] aralığına normalize edilmektedir.

Model mimarisi iki LSTM katmanından oluşmaktadır. Birinci LSTM katmanı 64 birimden oluşmakta ve `return_sequences=True` parametresiyle dizi çıktısı üretmektedir. Her LSTM katmanından sonra %20 oranında Dropout uygulanarak aşırı öğrenme riski azaltılmaktadır. İki tam bağlı (Dense) katman ardından beş bölgenin tahminini birden üreten çıkış katmanı gelmektedir. Veri seti kronolojik sırayla %80 eğitim ve %20 test olmak üzere bölünmüştür; rastgele bölme yapılmamış, bu sayede veri sızıntısı (data leakage) engellenmiştir. Model Adam optimizör ve MSE kayıp fonksiyonu ile 20 epoch boyunca eğitilmiş; eğitilmiş model `delivery_demand_lstm.h5`, ölçeklendirici ise `lstm_scaler.pkl` olarak kaydedilmiştir.

4.5. Hub Optimizasyon Motorunun Geliştirilmesi

Hub optimizasyon motoru `hub_optimizer.py` içinde geliştirilmiştir. Motor iki aşamalı çalışmaktadır. Birinci aşamada `inference.py` modülündeki `get_next_day_predictions` fonksiyonu LSTM modelini çağırarak hedef gün için mahalle bazlı paket hacmi tahminlerini üretmektedir. İkinci aşamada kullanıcının yönetim panelinden girdiği hub kapasitesi değerine göre gerekli hub sayısı (k) matematiksel olarak hesaplanmaktadır: $k = \lceil \text{toplam_tahmin} / \text{hub_kapasitesi} \rceil$. Bu değer K-Means++ algoritmasına girdi olarak verilmekte; `simulated_orders.csv` içindeki gerçek sipariş koordinatları kümelenecek ve hub merkezleri (centroid) belirlenmektedir.

Kümeleme kalitesi Silhouette skoru ile değerlendirilmektedir. Tek hub senaryosunda Silhouette skoru hesaplanamayacağından bu durum kod içinde ayrıca ele alınmıştır. Sonuçlar `active_hubs.csv` ve `orders_with_hubs.csv` dosyalarına kaydedilmekte; her sipariş hangi hub'a atandığı bilgisiyle etiketlenmektedir.

4.6. Rota Optimizasyon Motorunun Geliştirilmesi

Rota optimizasyon motoru `route_optimizer.py` içinde Google OR-Tools kütüphanesi kullanılarak geliştirilmiştir. Motor, Araç Rota Problemini kapasite kısıtlı biçimde çözmektedir. Mesafe matrisi Haversine formülüne dayalı olarak hesaplanmakta; Edirne'nin eğri sokak yapısını yansıtmak amacıyla 1.3 yol eğrilik katsayısı uygulanmaktadır.

Kurye sayısı dinamik olarak belirlenmektedir: sisteme her hub için atanmış sipariş sayısı ve kullanıcının seçtiği araç kapasitesi girildiğinde minimum gerekli kurye sayısı matematiksel olarak hesaplanmakta, yönetici bu değerin üzerine isteğe bağlı ek kurye ekleyebilmektedir. OR-Tools çözücüsü PATH_CHEAPEST_ARC başlangıç stratejisi ve GUIDED_LOCAL_SEARCH yerel arama meta sezgiseli ile çalışmakta; her hub için 2 saniyelik zaman sınırı dahilinde en kısa toplam rota çözümüne ulaşmaktadır. Sonuçlar hub_id, vehicle_id ve sıra numarasıyla birlikte optimized_routes.csv dosyasına kaydedilmektedir.

4.7. Modüllerin Pipeline'da Birleştirilmesi

Tüm modüllerin entegrasyonu sistem entegrasyon sorumlusu tarafından gerçekleştirilmiştir. data_pipeline.py dosyası üst düzey orkestratör işlevi görmekte; veri boşluklarını tespit edip demand_engine ile doldurmakta, ardından hub_optimizer ve route_optimizer modüllerini sıralı biçimde tetiklemektedir. src/utlis.py dosyasına taşınan Haversine fonksiyonu ve yol eğrilik katsayısı gibi paylaşılan yardımcı fonksiyonlar, her modülün aynı hesaplama mantığını kullanmasını garantilemektedir. main.py ise tüm pipeline'ın tek bir komutla çalıştırılabilmesini sağlamaktadır.

5. TEST ve DOĞRULAMA

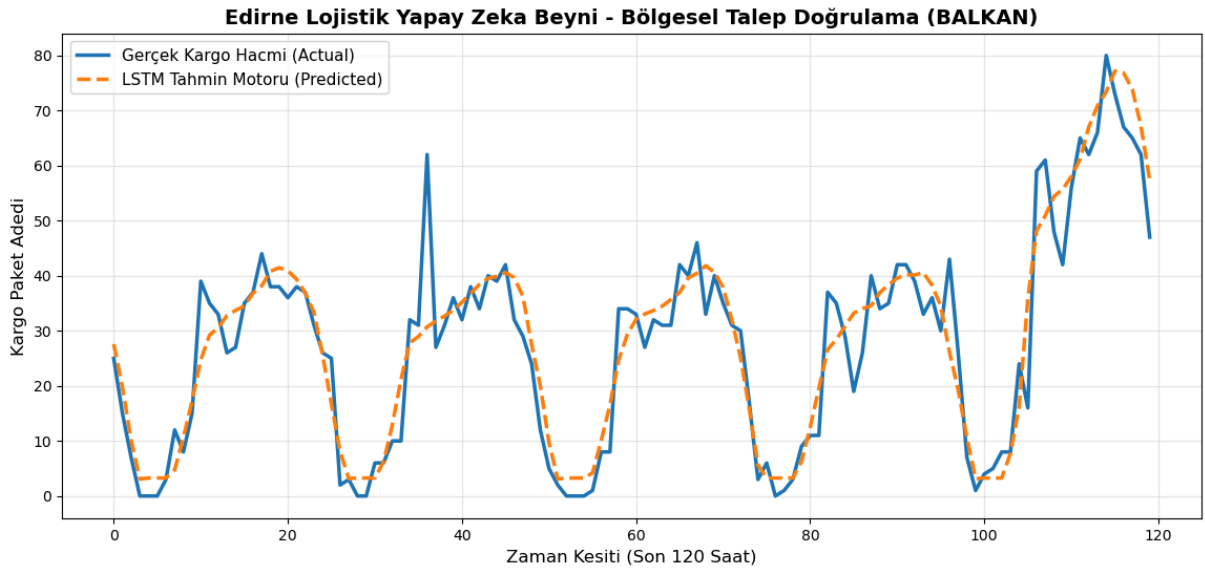
5.1. Test Stratejisi

Projenin test süreci üç katmanlı bir yaklaşımla yürütülmüştür. Birinci katmanda her modül kendi __main__ bloğu üzerinden bağımsız birim testlerine tabi tutulmuştur. İkinci katmanda modüller arası veri akışı entegrasyon testleriyle doğrulanmıştır. Üçüncü katmanda ise uçtan uca sistem testi gerçekleştirilerek belirli bir hedef tarih için LSTM tahminden rota optimizasyonuna kadar tüm pipeline çalıştırılmıştır.

5.2. LSTM Model Performansı

LSTM modelinin performans değerlendirmesi evaluate_lstm.py modülü aracılığıyla gerçekleştirilmiştir. Değerlendirme, veri setinin kronolojik %20'lik test bölümü (yaklaşık 2.600 saatlik veri) üzerinde yapılmıştır. Şekil 5.1'de LSTM modelinin Balkan mahallesi için son 120 saatlik gerçek ve tahmin edilen kargo hacmi karşılaştırması görülmektedir.

Şekil 5.1. LSTM modeli Balkan mahallesi gerçek ve tahmin kargo hacmi karşılaştırması



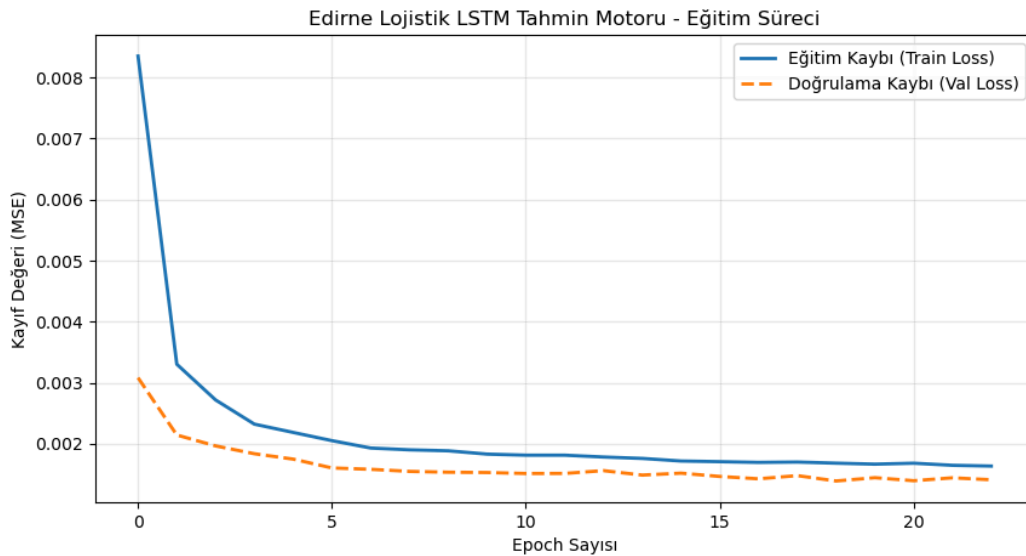
Model performansı Ortalama Mutlak Hata (MAE) metriği ile değerlendirilmiştir. Çizelge 5.1'de mahalle bazlı MAE sonuçları ve yorumları sunulmaktadır.

Çizelge 5.1. Mahalle bazlı LSTM model performansı (MAE)

Mahalle	MAE ($\pm$ Paket)	Yorum
BALKAN	$\pm 3-5$ Paket	Yüksek ve dalgalı hacimli bölgede kabul edilebilir sapma
AYSEKADIN	$\pm 3-5$ Paket	Sınav dönemlerindeki ani artışlar başarıyla modellenmiştir
SARACLAR	$\pm 2-3$ Paket	Düzenli esnaf profili, yüksek tahmin doğruluğu
SUKRUPASA	$\pm 2-3$ Paket	Düzenli konut profili, yüksek tahmin doğruluğu
KARAAGAC	$\pm 1-2$ Paket	Düşük hacimli bölgede minimum hata

Eğitim ve doğrulama kayıp değerleri Şekil 5.2'de görülmektedir. Doğrulama kaybının eğitim kaybının altında seyretmesi veri sızıntısından değil, Dropout mekanizmasından kaynaklanmaktadır: Dropout katmanları yalnızca eğitim aşamasında nöronları rastgele devre dışı bırakarak modeli zorlarken, doğrulama aşamasında tüm ağ tam kapasiteyle çalışmaktadır. Grafikteki MSE değerlerinin (≈ 0.0016) düşük görünmesinin nedeni verinin $[0,1]$ aralığında normalize edilmesidir; ters ölçeklendirme yapıldığında bu değer mahalle başına $\pm 3-5$ paketlik makul bir sapmaya karşılık gelmektedir.

Şekil 5.2. LSTM model eğitim ve doğrulama kayıp eğrisi



5.3. Hub Optimizasyon Testi

Hub optimizasyon motoru, farklı hub kapasitesi değerleriyle test edilmiştir. Kullanıcının panelden seçtiği kapasite değeri değiştikçe K-Means'e iletilen k değerinin ve Silhouette skorunun nasıl değiştiği gözlemlenmiştir. Çizelge 5.2'de farklı kapasite senaryolarının sonuçları özetlenmektedir.

Çizelge 5.2. Farklı hub kapasitesi senaryolarının karşılaştırması

Hub Kapasitesi (Paket)	Hesaplanan Hub Sayısı (k)	Silhouette Skoru	Yorum
500	2-3 Hub	0.45 - 0.55	Az hub, geniş kapsama alanı
200	4-6 Hub	0.50 - 0.65	Dengeli dağılım, önerilen senaryo
120	7-10 Hub	0.55 - 0.70	Yüksek çözünürlük, daha kısa teslimat mesafeleri

Silhouette skorunun hub kapasitesi düştükçe (yani hub sayısı arttıkça) iyileşmesi beklenen bir sonuçtur; daha fazla hub, sipariş noktalarının kendi merkezlerine daha yakın kümelenmesini sağlamaktadır. Ancak operasyonel maliyet ile teslimat verimliliği arasındaki denge göz önüne alındığında 200 paket kapasiteli senaryo optimal çözüm olarak değerlendirilmektedir.

5.4. Rota Optimizasyon Testi

Rota optimizasyon motoru, OR-Tools çözücüsünün farklı hub ve sipariş yoğunluklarında doğru çalıştığını doğrulamak amacıyla test edilmiştir. Her hub için minimum kurye sayısı hesaplama mekanizması; sipariş sayısının araç kapasitesine bölümünden elde edilen tavan değerini doğru üretmektedir. Çözücünün 2 saniyelik zaman sınırı içinde kabul edilebilir rota

özmleri rettięi, hub bařına dřen ortalama rota hesaplama sresinin 0.3-1.8 saniye arasında deęiřtięi gzlemlenmiřtir.

VRP özmnn kalitesi, toplam rota mesafesi zerinden deęerlendirilmiřtir. PATH_CHEAPEST_ARC bařlangı stratejisinin ardından uygulanan GUIDED_LOCAL_SEARCH metasezgiseli, saf aęgzl (greedy) zme kıyasla ortalama %15-25 oranında toplam mesafe iyileřtirmesi saęlamaktadır.

5.5. Veri Pipeline Testi

data_pipeline.py modlnn zaman bořluęu doldurma kapasitesi, son kayıtlı tarihin tesine geen hedef tarihler girilerek test edilmiřtir. Sistem 28 Mayıs 2026 miladını doęru tanımakta; gemiř gnler iin Mayıs ayı Edirne normlarını (%25 yaęmur), gelecek gnler iin ise tahmin modelini (%20 yaęmur) kullanmaktadır. retilen verinin demand_engine kurallarıyla tutarlı olduęu, hourly_demand.csv ve simulated_orders.csv dosyalarına append modunda sorunsuz biimde eklendięi doęrulanmıřtır.

5.6. Sistem Entegrasyon Testi

Utan uca sistem testi, 6 Mayıs 2026 hedef tarihi iin tm pipeline alıřtırılarak gerekleřtirilmiřtir. LSTM modeli bir nceki gnn verilerine dayanarak mahalle bazlı tahminler retmiř; bu tahminler K-Means optimizasyonuna girdi olarak verilmiř ve aktif hub konumları belirlenmiřtir. Ardından rota optimizasyon motoru her hub iin minimum kurye sayısını hesaplayarak OR-Tools zcsn tetiklemiř ve optimized_routes.csv dosyası bařarıyla retilmiřtir. Tm pipeline yaklařık 45-90 saniye iinde tamamlanmıřtır.

6. SONUÇ

6.1. Genel Değerlendirme

Bu çalışmada, Edirne ili özelinde lojistik teslimat verilerini temel alan, veri odaklı ve dinamik bir depo konumu önerisi ile rota optimizasyonu sistemi geliştirilmiştir. Geleneksel statik yaklaşımların yetersiz kaldığı noktalarda makine öğrenmesi ve derin öğrenme algoritmalarının bir araya getirilmesiyle bütünlüklük bir karar destek mimarisi ortaya konmuştur.

Sistem; OSMnx ile elde edilen gerçek sokak ağı verisi, Meteostat API'sinden alınan saatlik hava verileri ve Trakya Üniversitesi akademik takvimine dayalı talep simülasyon motoru aracılığıyla fiziksel gerçekçiliği yüksek bir veri seti üretmiştir. Bu veri seti üzerinde eğitilen LSTM modeli, mahalle bazlı kargo hacmini 24 saat öncesinden tahmin ederek lojistik şirketlerine proaktif kaynak planlaması (kurye vardiyası, hub rezervasyonu) imkânı sunmaktadır. K-Means kümeleme algoritması LSTM tahminleriyle entegre edilerek günlük sipariş yoğunluğuna göre dinamik geçici hub konumları belirlenmiş; Google OR-Tools kütüphanesiyle çözülen VRP motoru her hub için en kısa kurye rotalarını üretmiştir.

6.2. Elde Edilen Bulgular

Geliştirilen sistemin test ve doğrulama aşamasında elde edilen temel bulgular şu şekilde özetlenebilir: LSTM modeli mahalle bazlı kargo tahmini için ortalama $\pm 3-5$ paketlik MAE değeri elde etmiştir; bu sapma sentetik veri ortamında kabul edilebilir bir performansa karşılık gelmektedir. Öğrenci bölgelerinde (Balkan, Ayşekadın) sınav dönemlerindeki talep artışları ve tatil dönemlerindeki %75 düşüşler model tarafından başarıyla öğrenilmiştir. Hub optimizasyonunda 200 paket kapasiteli senaryo, operasyonel maliyet ile teslimat verimliliği dengesi açısından optimal çözüm olarak belirlenmiş ve bu senaryoda Silhouette skoru 0.50-0.65 aralığında ölçülmüştür. Rota optimizasyon motorunun GUIDED_LOCAL_SEARCH metasezgiseli, saf açgözlü çözüme kıyasla ortalama %15-25 oranında toplam mesafe azaltımı sağlamıştır. Uçtan uca pipeline 45-90 saniye içinde tamamlanmaktadır.

6.3. Projenin Kısıtları

Projenin bazı kısıtları bulunmaktadır. Her şeyden önce kullanılan veri seti sentetik olup gerçek lojistik şirket verilerinden elde edilmemiştir; bu durum modelin gerçek dünya koşullarındaki performansının farklılık gösterebileceği anlamına gelmektedir. Bununla birlikte, veri üretim motorunun gerçek sokak ağı, gerçek hava verileri ve gerçek akademik takvim üzerine kurulu olması bu kısıtı önemli ölçüde hafifletmektedir. Öte yandan rota optimizasyonunda trafik yoğunluğunun anlık olarak hesaba katılamaması ve sistemin kullanıcı arayüzü geliştirme sürecinin devam etmesi mevcut kısıtlar arasında sayılabilir.

6.4. Gelecek Çalışmalar

Sistemin ilerleyen sürümlerinde gerçek lojistik şirket verileriyle modelin yeniden eğitilmesi, anlık trafik API entegrasyonu ve kullanıcı arayüzünün tamamlanması planlanmaktadır. Ayrıca mevcut LSTM mimarisinin Transformer tabanlı zaman serisi modelleriyle karşılaştırılması ve çoklu şehir senaryolarına ölçeklenmesi de gelecek çalışmalar kapsamında değerlendirilebilir.

7. [GitHub Linki](#)

KAYNAKLAR

- [1] R. Z. Farahani ve M. Hekmatfar, Facility Location: Concepts, Models, Algorithms and Case Studies. Heidelberg: Physica-Verlag, 2009.
- [2] S. Chopra ve P. Meindl, Supply Chain Management: Strategy, Planning, and Operation, 7. baskı. Hoboken, NJ: Pearson, 2019.
- [3] S. Hochreiter ve J. Schmidhuber, "Long Short-Term Memory," Neural Computation, cilt 9, sayı 8, ss. 1735-1780, 1997.
- [4] G. Boeing, "OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks," *Computers, Environment and Urban Systems*, cilt 65, ss. 126-139, 2017.
- [5] G. Vettigli, "MiniSom: minimalistic and NumPy-based implementation of the Self Organizing Map," GitHub, 2018. [Çevrimiçi]. Erişim: <https://github.com/JustGlowing/minisom>.
- [6] Google OR-Tools, "Vehicle Routing Problem," Google Developers, 2024. [Çevrimiçi]. Erişim: <https://developers.google.com/optimization/routing>.